

Spring — Assessment A1

Answer Key — Theme: Campus Connect Portal

SECTION A — Conceptual Application

Q1. Explain the flow of a request in Spring MVC from JSP form to Controller to database and back to view.

A browser submits a JSP form (e.g., a login form) to a URL. Every request first hits the DispatcherServlet, Spring MVC's single front controller. The DispatcherServlet consults a HandlerMapping to find which @Controller method matches the requested URL and HTTP method, then a HandlerAdapter invokes that method. The controller method extracts the form data (bound automatically into a model/command object), calls a Service, which calls a DAO/Repository to read or write to the database via JDBC/JPA. The controller then adds results to the Model and returns a logical view name (a String, e.g., "dashboard") instead of a file path. The DispatcherServlet passes that name to a ViewResolver (e.g., InternalResourceViewResolver), which maps it to a physical JSP under /WEB-INF/views/. Finally the JSP is rendered with the Model's data and the resulting HTML is sent back to the browser.

Logic:

- The DispatcherServlet is the single entry point for every request, which is what lets Spring MVC centralize cross-cutting concerns (mapping, view resolution, exception handling) instead of scattering them across servlets.
- Separating the 'logical view name' from the 'physical JSP path' (via ViewResolver) means a controller never hardcodes a file path, so the same controller logic can be reused with a different view technology later.
- The database access always happens inside the controller-called Service/DAO layer, never inside the JSP — the JSP's only job is to render whatever the Model already contains.

Q2. Explain IoC (Inversion of Control) and Dependency Injection in Spring.

In ordinary object-oriented code, a class creates its own dependencies with new (e.g., a Controller does new UserDAO()), meaning the class controls its own object graph. Inversion of Control flips this: the Spring IoC container creates, configures, and wires objects (called beans), and the class simply declares what it needs. Dependency Injection (DI) is the specific mechanism Spring uses to achieve IoC — the container 'injects' a bean's required collaborators (via constructor or setter) rather than the bean instantiating them itself.

Logic:

- Because the container owns object creation, a bean's dependencies can be swapped (e.g., a mock UserDAO in a test) without changing the bean's own code — this is the main testability/flexibility benefit of IoC.
- @Autowired (or constructor parameters, in modern Spring) is the annotation that tells the container which bean to inject where; the container resolves it by type (and by @Qualifier when there are multiple candidates).
- 'Control' here refers to control over the object's lifecycle and wiring — it has been inverted from the object itself to the Spring container.

Q3. Differentiate between BeanFactory and ApplicationContext.

Both are Spring IoC containers, but `ApplicationContext` is a superset of `BeanFactory` with much more built-in functionality, and it is what nearly every real Spring application uses. `BeanFactory` follows lazy initialization by default — a bean is only created the first time it is requested with `getBean()`.

`ApplicationContext` eagerly initializes singleton beans as soon as the context starts, catching wiring errors immediately at startup rather than later when a bean is first used.

Logic:

- `ApplicationContext` additionally provides AOP integration, event publication (`ApplicationEvent/ApplicationListener`), internationalization via `MessageSource`, and easy integration with web environments (`WebApplicationContext`) — none of which `BeanFactory` provides directly.
- Because `ApplicationContext` eagerly creates singleton beans, a misconfigured bean (e.g., a missing dependency) fails fast at application startup instead of surfacing as a runtime error mid-request, which is generally preferred in production systems.
- `BeanFactory` is mostly used in memory-constrained environments where lazy loading matters; virtually all Spring MVC/Boot applications use an `ApplicationContext` implementation such as `AnnotationConfigWebApplicationContext`.

Q4. Explain how `JdbcTemplate` simplifies JDBC operations.

Raw JDBC requires manually opening a `Connection`, creating a `Statement/PreparedStatement`, executing it, iterating a `ResultSet`, mapping rows to objects, and closing every resource in a finally block — and converting checked `SQLExceptions` into something usable. `JdbcTemplate` is a Spring helper class that wraps all of this: the developer supplies only the SQL string, the parameters, and (when a result is expected) a `RowMapper` describing how to turn one row into one object; `JdbcTemplate` handles acquiring and releasing the `Connection/Statement/ResultSet` automatically, even when an exception occurs.

Logic:

- `JdbcTemplate` translates every `SQLException` into one of Spring's unchecked `DataAccessException` subclasses, so calling code isn't forced to catch a checked exception it usually can't recover from.
- `queryForObject(sql, rowMapper, params)` and `update(sql, params)` reduce a typical multi-line JDBC boilerplate block down to a single method call.
- Because resource cleanup is guaranteed by the template itself, a developer can never accidentally forget to close a `Connection` or `ResultSet` — the classic source of JDBC connection leaks.

Q5. Explain the role of `DispatcherServlet` in Spring MVC.

`DispatcherServlet` implements the Front Controller design pattern for Spring MVC: it is the one servlet registered to receive every incoming HTTP request for the application (or a URL pattern such as `/`). It does not contain any business logic itself; instead it orchestrates the whole request lifecycle — consulting `HandlerMapping` to find the right controller method, invoking it through a `HandlerAdapter`, receiving back a `ModelAndView`, passing the logical view name to a `ViewResolver`, and finally rendering the resolved View with the Model's data.

Logic:

- Centralizing routing in one servlet means features like global exception handling (`@ControllerAdvice`), consistent view resolution, and request interceptors (`HandlerInterceptor`) can be configured once and apply to every controller in the application.
- `DispatcherServlet` is configured either in `web.xml` (mapped to a URL pattern with an associated context config file) or, in Java-based config, via a class extending `AbstractAnnotationConfigDispatcherServletInitializer`, which Spring auto-detects on startup.

Q6. Explain constructor injection vs setter injection.

Constructor injection supplies a bean's required dependencies as parameters to its constructor, so the object can never exist in a partially-wired state — once constructed, all its dependencies are guaranteed to be present and the fields can be declared final. Setter injection instead supplies dependencies through public setter methods after the object has already been constructed with a no-arg constructor, meaning the dependency is optional and can be reassigned or left unset.

Logic:

- Constructor injection is generally preferred for mandatory dependencies because it makes immutability possible (final fields) and makes a missing dependency a compile-time/startup-time error rather than a `NullPointerException` discovered later at runtime.
- Setter injection suits optional dependencies or configuration that might need to change after construction (e.g., a reconfigurable timeout value), and it can also help break certain circular-dependency situations that pure constructor injection cannot resolve.
- Modern Spring (and Spring Boot) documentation recommends constructor injection as the default choice, reserving setter injection for genuinely optional properties.

SECTION B — Practical Tasks

QB1. Spring MVC Configuration — configure DispatcherServlet and application context.

Either approach registers the same front controller; the XML approach declares it in web.xml and points to XML bean-definition files, while the Java-based approach uses `AbstractAnnotationConfigDispatcherServletInitializer`, which Spring detects automatically on Servlet 3.0+ containers without any web.xml at all.

```
<!-- web.xml approach -->
<servlet>
<servlet-name>dispatcher</servlet-name>
<servlet-class>org.springframework.web.servlet.DispatcherServlet</servlet-class>
<init-param>
<param-name>contextConfigLocation</param-name>
<param-value>/WEB-INF/spring-mvc-config.xml</param-value>
</init-param>
<load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
<servlet-name>dispatcher</servlet-name>
<url-pattern>/</url-pattern>
</servlet-mapping>

// Java-based approach
public class AppInitializer extends AbstractAnnotationConfigDispatcherServletInitializer {
@Override protected Class<?>[] getRootConfigClasses() { return new Class<?>[]{ RootConfig.class };
}
@Override protected Class<?>[] getServletConfigClasses() { return new Class<?>[]{ WebConfig.class
}; }
@Override protected String[] getServletMappings() { return new String[]{ "/" }; }
}
```

Logic:

- `getRootConfigClasses()` supplies configuration shared across the whole application (services, DAOs), while `getServletConfigClasses()` supplies web-layer-only configuration (controllers, view resolvers) — keeping the two contexts cleanly separated.
- `getServletMappings()` returning "/" tells the DispatcherServlet to handle every request path, which is what makes it the single front controller for the app.

QB2. LoginController Implementation — bind form data and handle login requests.

`@Controller` marks the class as a Spring MVC handler bean (returning view names, unlike `@RestController`). A `@GetMapping("/login")` method displays the empty form; a `@PostMapping("/login")` method receives the submitted fields bound automatically into a `@ModelAttribute` command object, whose field names Spring matches to the JSP form's input names.

```
@Controller
public class LoginController {
@Autowired private UserDao userDao;

@GetMapping("/login")
public String showForm(Model model) {
model.addAttribute("loginForm", new LoginForm());
return "login";
}

@PostMapping("/login")
public String handleLogin(@ModelAttribute("loginForm") LoginForm form, Model model) {
boolean valid = userDao.validateUser(form.getUsername(), form.getPassword());
if (!valid) {
model.addAttribute("error", "Invalid username or password");
return "login";
}
}
```

```
return "redirect:/dashboard";
}
}
```

Logic:

- `@ModelAttribute("loginForm")` LoginForm form both binds request parameters onto the object and makes that same object available to the view under the same name, so the JSP's `<form:form modelAttribute="loginForm">` tag can pre-populate or redisplay it on error.
- Returning a logical view name string ("dashboard" or "login") rather than an object keeps the controller decoupled from exactly which JSP file renders it.

QB3. JdbcTemplate & DAO Layer — validate credentials against the database.

UserDAO holds a JdbcTemplate field injected by Spring. `validateUser()` runs a parameterized `SELECT COUNT(*)` query with the username and password as bind parameters and returns true only if exactly one matching row exists — all `Connection/PreparedStatement/ResultSet` handling is delegated to JdbcTemplate.

```
@Repository
public class UserDAO {
    private final JdbcTemplate jdbcTemplate;
    public UserDAO(JdbcTemplate jdbcTemplate) { this.jdbcTemplate = jdbcTemplate; }

    public boolean validateUser(String username, String password) {
        String sql = "SELECT COUNT(*) FROM users WHERE username = ? AND password = ?";
        Integer count = jdbcTemplate.queryForObject(sql, Integer.class, username, password);
        return count != null && count == 1;
    }
}
```

Logic:

- Using `?` placeholders with `queryForObject(sql, Integer.class, username, password)` prevents SQL injection in exactly the same way `PreparedStatement` does in raw JDBC — JdbcTemplate uses `PreparedStatement` internally.
- `queryForObject` returning an Integer count (0 or 1) avoids needing a full `RowMapper` for a query that only needs a yes/no answer.
- No explicit `close()` calls are written anywhere in this DAO — Spring's JdbcTemplate guarantees resource cleanup even if the query throws.

QB4. View Resolution (JSP) — map logical view names to physical JSP files.

An `InternalResourceViewResolver` bean is declared once in the web configuration, with a prefix (`"/WEB-INF/views/"`) and a suffix (`".jsp"`). When a controller returns "dashboard", the resolver concatenates prefix + viewName + suffix to locate `/WEB-INF/views/dashboard.jsp` and forwards the request to it internally.

```
@Bean
public ViewResolver viewResolver() {
    InternalResourceViewResolver resolver = new InternalResourceViewResolver();
    resolver.setPrefix("/WEB-INF/views/");
    resolver.setSuffix(".jsp");
    return resolver;
}
```

Logic:

- Placing JSPs under `/WEB-INF/` (rather than a publicly accessible path) prevents users from requesting the raw JSP file directly by URL — they can only reach it by going through a controller, which is a standard security practice.

- Because the mapping is centralized in one bean, changing the JSP folder structure later only requires editing the resolver's prefix, not every controller method.

QB5. Custom Error Handling — provide user-friendly feedback for invalid credentials.

Two valid approaches: (1) the controller itself detects a failed `validateUser()` call and returns to the login view with an error message added to the Model (as shown in B2), or (2) a checked/unchecked `InvalidCredentialsException` is thrown from the DAO/Service layer and caught centrally by an `@ExceptionHandler` method (either inside the controller or in a class-wide `@ControllerAdvice`), which resolves to a dedicated error view.

```
public class InvalidCredentialsException extends RuntimeException {
    public InvalidCredentialsException(String message) { super(message); }
}

@Controller
public class LoginController {
    @PostMapping("/login")
    public String handleLogin(@ModelAttribute LoginForm form) {
        if (!userDAO.validateUser(form.getUsername(), form.getPassword())) {
            throw new InvalidCredentialsException("Invalid username or password");
        }
        return "redirect:/dashboard";
    }

    @ExceptionHandler(InvalidCredentialsException.class)
    public ModelAndView handleInvalidLogin(InvalidCredentialsException ex) {
        ModelAndView mav = new ModelAndView("error");
        mav.addObject("message", ex.getMessage());
        return mav;
    }
}
```

Logic:

- Handling the failure inline (returning "login" with `model.addAttribute("error", ...)`) keeps the user on the same form so they can immediately retry, which is usually the friendlier UX for a login failure.
- `@ExceptionHandler(InvalidCredentialsException.class)` centralizes error-to-view mapping so every controller that might throw that exception gets the same consistent `error.jsp` response without repeating try/catch logic.

SECTION C — Mini Project: Campus Connect Portal

QC1. Student Registration & Login — Spring Form Binding + Spring Data JPA persistence.

A Student entity is mapped with `@Entity/@Id/@GeneratedValue`, backed by a `StudentRepository` extending `JpaRepository<Student, Long>`. `RegistrationController` exposes a GET handler that puts an empty `Student` into the Model, and a POST handler bound via Spring's `<form:form modelAttribute="student">` JSP tag, which automatically populates the `Student` object's fields from the submitted form and saves it via the repository.

```
@Entity
public class Student {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String email;
    private String password;
    // getters/setters
}

public interface StudentRepository extends JpaRepository<Student, Long> { }

@Controller
public class RegistrationController {
    @Autowired private StudentRepository studentRepository;

    @GetMapping("/register")
    public String showForm(Model model) {
        model.addAttribute("student", new Student());
        return "register";
    }

    @PostMapping("/register")
    public String register(@ModelAttribute("student") Student student) {
        studentRepository.save(student);
        return "redirect:/login";
    }
}
```

Logic:

- Spring's form-tag library (`<form:input path="name"/>`, etc.) binds each input directly to a property of the `modelAttribute` object by matching the path to a getter/setter, so no manual `request.getParameter()` calls are needed.
- Because `StudentRepository` extends `JpaRepository`, `save(student)` is enough to both insert a new student (registration) and update an existing one, with no SQL written by the developer.

QC2. Event Creation & Validation — Hibernate Validator enforces non-empty titles and future dates.

The `Event` entity's fields carry Bean Validation annotations: `@NotBlank` on `title`, `@Future` on `eventDate`. The controller's `create` method annotates the incoming `Event` with `@Valid`, and a `BindingResult` parameter immediately following it captures any constraint violations so the controller can re-render the form with error messages instead of saving invalid data.

```
@Entity
public class Event {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Event title cannot be empty")
    private String title;

    @Future(message = "Event date must be in the future")
    private Date eventDate;
}
```

```

private LocalDate eventDate;
// getters/setters
}

@Controller
public class EventController {
@Autowired private EventRepository eventRepository;

@PostMapping("/events/create")
public String create(@Valid @ModelAttribute("event") Event event, BindingResult result) {
if (result.hasErrors()) {
return "event-form";
}
eventRepository.save(event);
return "redirect:/events";
}
}

```

Logic:

- @Valid triggers Hibernate Validator (the default Bean Validation provider) automatically when Spring binds the form data onto the Event object, before the controller body's own logic runs.
- The BindingResult parameter must be declared directly after the @Valid object in the method signature — if it is omitted, a validation failure throws an exception instead of being caught gracefully.
- Checking bindingResult.hasErrors() before calling eventRepository.save(...) guarantees an invalid event (blank title or a past date) never reaches the database.

QC3. Event Listing & Thymeleaf — display upcoming events in a structured HTML grid.

A GET /events handler fetches all events via EventRepository.findAll() (optionally filtered/sorted by date) and adds the list to the Model. The Thymeleaf template iterates over it with th:each, rendering one grid cell/row per event and reading each event's fields with th:text.

```

@Controller
public class EventListController {
@Autowired private EventRepository eventRepository;

@GetMapping("/events")
public String list(Model model) {
model.addAttribute("events", eventRepository.findAll());
return "event-list";
}
}

<!-- event-list.html -->
<div class="grid">
<div class="card" th:each="event : ${events}">
<h3 th:text="${event.title}">Title</h3>
<p th:text="${event.eventDate}">Date</p>
</div>
</div>

```

Logic:

- th:each="event : \${events}" repeats the enclosing HTML element once per list item, which is Thymeleaf's equivalent of a for-each loop directly inside the template.
- Thymeleaf templates are valid HTML even before processing (natural templating), so designers can preview event-list.html directly in a browser with placeholder content, unlike raw JSP scriptlets.

QC4. Session-Based Access Control — guard event creation behind a logged-in HttpSession.

On successful login, the controller stores the authenticated student in the HttpSession (session.setAttribute("user", student)). Any handler for a sensitive route, such as showing or processing

the event-creation form, first checks `session.getAttribute("user")` — if it is null, the user is redirected to `/login` instead of being allowed to proceed.

```
@Controller
public class EventController {
    @GetMapping("/events/new")
    public String showCreateForm(HttpSession session, Model model) {
        if (session.getAttribute("user") == null) {
            return "redirect:/login";
        }
        model.addAttribute("event", new Event());
        return "event-form";
    }

    @PostMapping("/events/create")
    public String create(@Valid @ModelAttribute("event") Event event, BindingResult result,
        HttpSession session) {
        if (session.getAttribute("user") == null) {
            return "redirect:/login";
        }
        if (result.hasErrors()) return "event-form";
        eventRepository.save(event);
        return "redirect:/events";
    }
}
```

Logic:

- Accepting `HttpSession` as a method parameter lets Spring MVC inject the current request's session directly, without manually pulling it from `HttpServletRequest`.
- Checking the session at the very top of the handler (before any business logic executes) ensures an unauthenticated request can never reach the code that would create an event, matching the requirement to redirect unauthenticated users to the login screen.
- For an application with many protected routes, this same check is often factored into a `HandlerInterceptor` so it doesn't need to be repeated in every controller method.